MATA60 - Prática de Banco de Dados EMPRESA - Navathe



Este material tem o objetivo geral de apresentar os impactos dos diferentes níveis de abstração oferecidos pelos modelos de entidade-relacionamento (MER) e relacional (ER). A modelagem, desde os níveis conceitual e lógico (providos pelo MER) e físico (oferecido pelo ER) são apresentados pelo autor Navathe no livro presente em nossa bibliografia básica.

1. Revisão da notação do MER
2. Modelagens para o banco de dados “EMPRESA”
    - **2.1. Modelo lógico**
    - **2.2 Modelo conceitual (convertido usando o BrModelo)**
    - **2.3 Modelo relacional (página 149 do livro)**
- **3. Criando o modelo físico (SQL-DDL)**
    - 3.1. DDL: alternativa 1
        1. Tabela `FUNCIONARIO`
        2. Tabela `DEPARTAMENTO`
        3. Tabela `LOCALIZACOES_DEPARTAMENTO`
        4. Tabela `DEPENDENTE`
        5. Tabela `PROJETO`
        6. Tabela `TRABALHA_EM`
    - 3.2. DDL: alternativa 2 - modularizando as constraints

        1. Criação das Tabelas Sem *Constraints*
        2. Adicionando *Constraints* Separadamente
4. DML - Inserindo dados
5. DML - primeiras consultas
6. ‘Views’ e ‘Materialized Views’
    Views
        CREATE VIEW
        CREATE OR REPLACE VIEW
        PARAMETER 'TEMPORARY’ or ‘TEMP’
        PARAMETER ‘WITH’ [CASCADED | LOCAL] CHECK OPTION
    Materialized Views
        CREATE MATERIALIZED VIEW IF NOT EXISTS
    REFRESH MATERIALIZED VIEW
7. Stored procedures
    CREATE PROCEDURE
    CALL
Transações
    Demonstrando a exclusão mútua
    Pontos de salvamento

1. Revisão da notação do MER

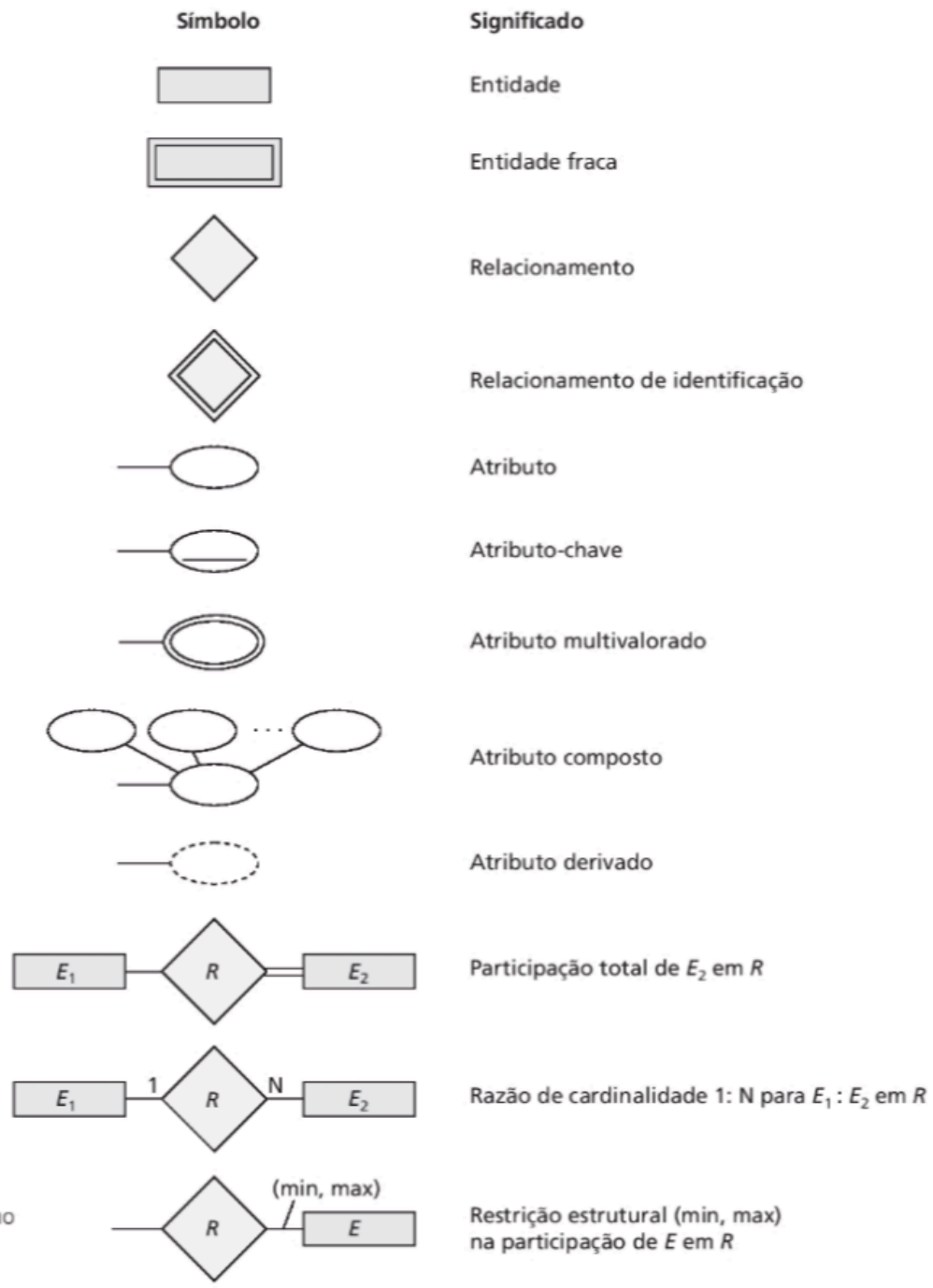


Figura 3.14 Resumo da notação para diagramas ER.

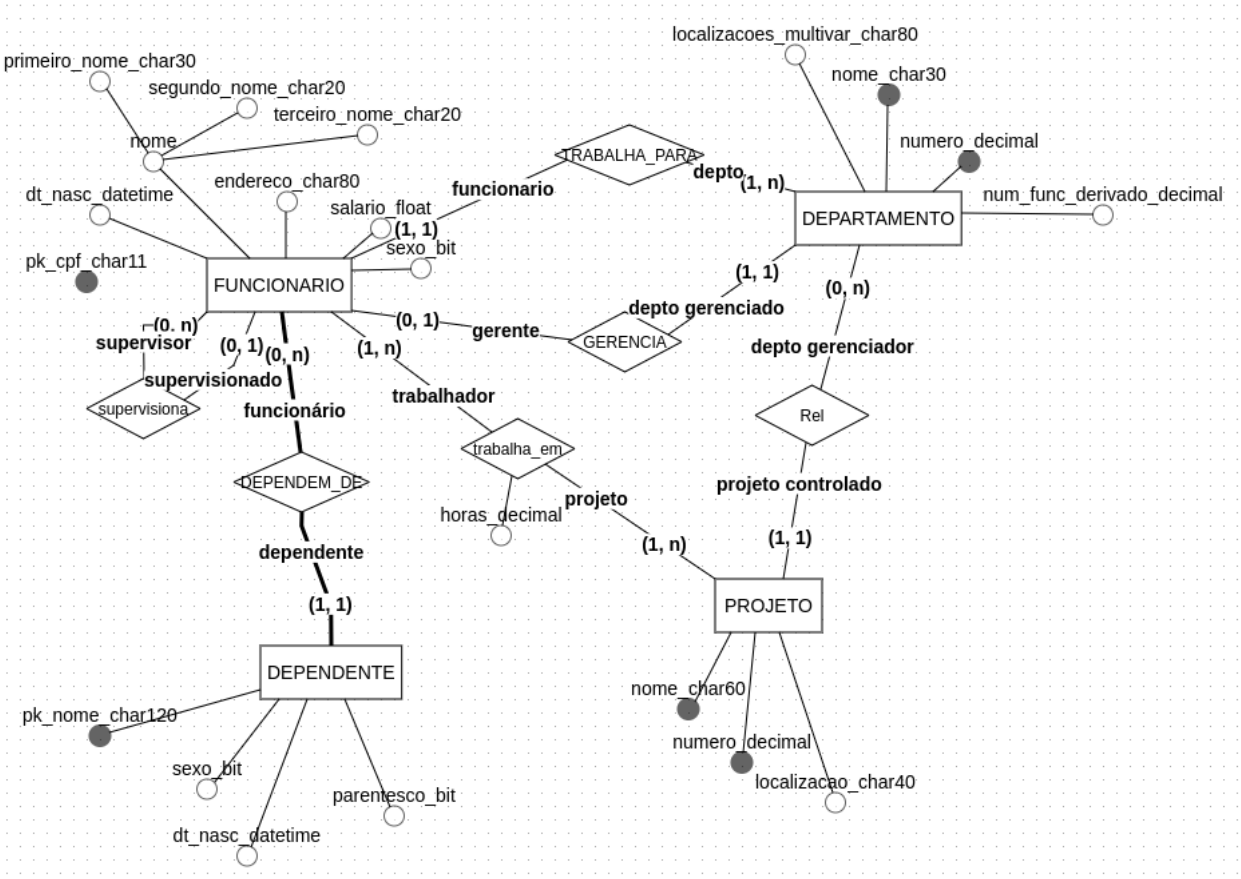
[MINIMUNDO] Descrição da EMPRESA

2. Modelagens para o banco de dados “EMPRESA”

2.1. Modelo lógico

BRMW

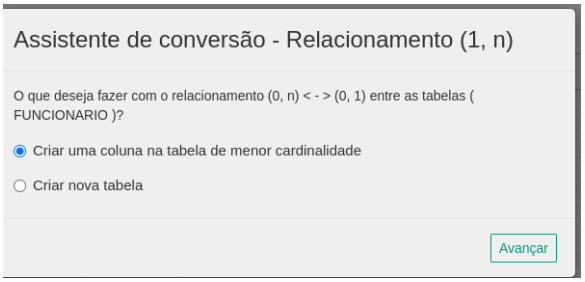
https://app.brmodeloweb.com/#!/publicview/672a6ec512523ba1aebd1122

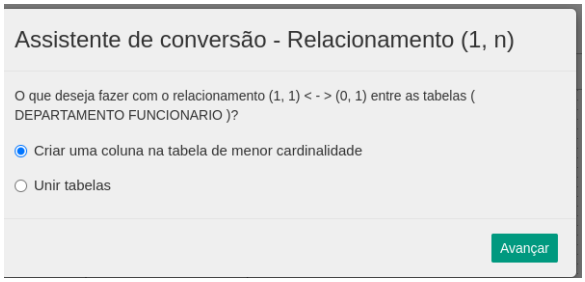


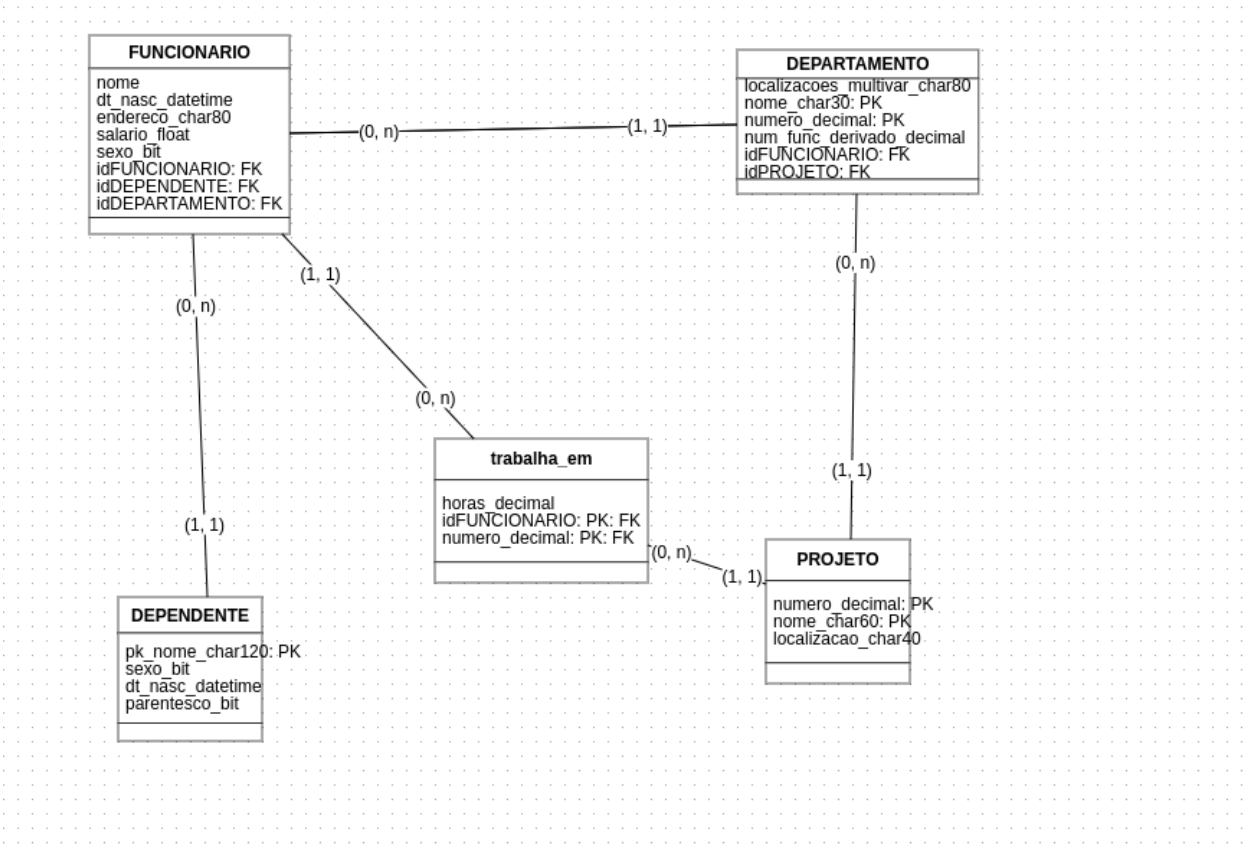
2.2 Modelo conceitual (convertido usando o BrModelo)

BRMW

https://app.brmodeloweb.com/#!/publicview/672a703312523ba1aebd11d1







2.3 Modelo relacional (página 149 do livro)

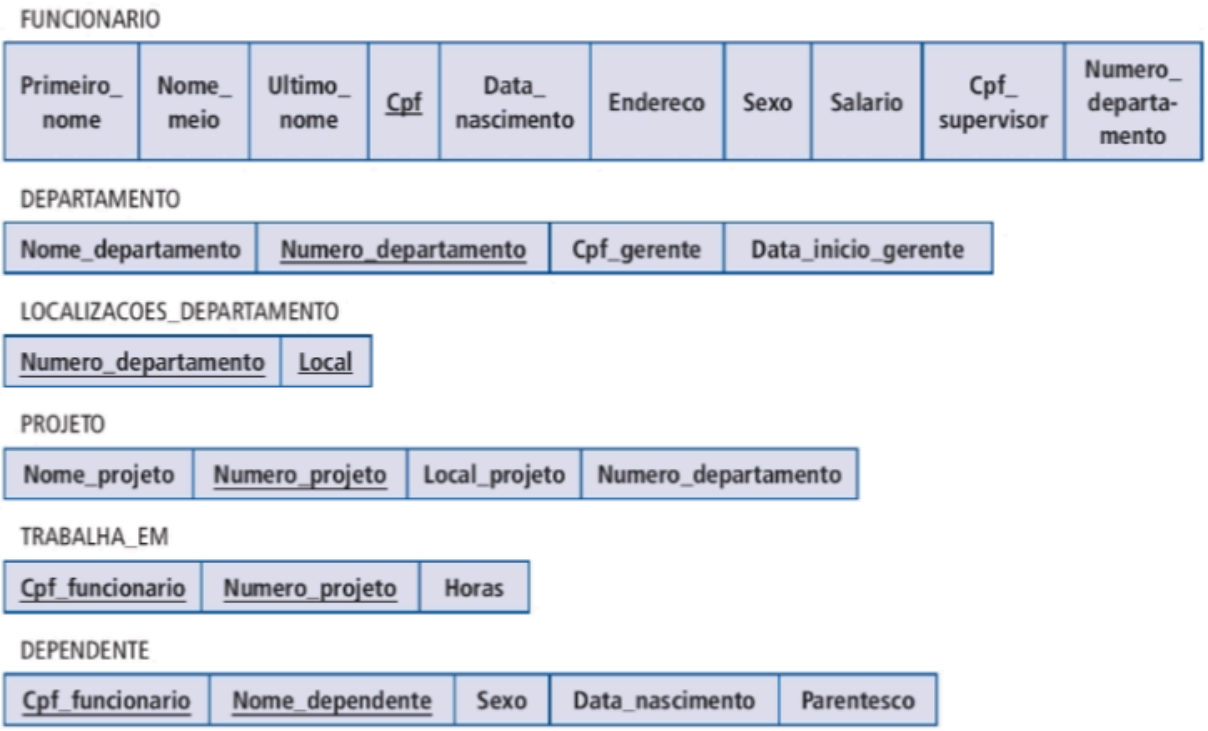


Figura 5.5 Diagrama de esquema para o esquema de banco de dados relacional EMPRESA.

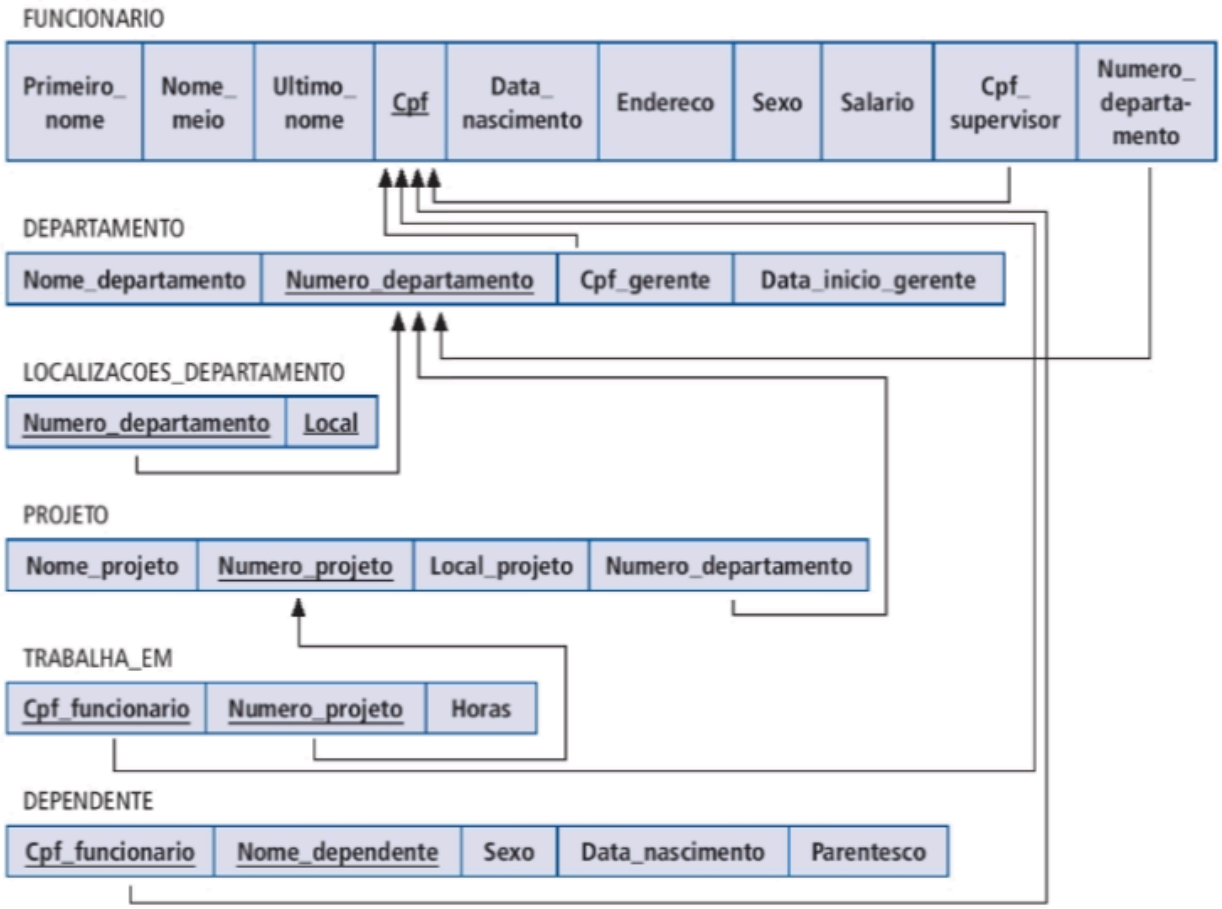


Figura 5.7 Restrições de integridade referencial exibidas no esquema de banco de dados relacional EMPRESA.

```
/*
Com erro
INSERT INTO DEPARTAMENTO (Nome_departamento, Numero_departamento,
Cpf_gerente, Data_inicio_gerente)
VALUES
('Pequisa', 5, '33344555587', '1988-05-22'),
('Administração', 4, '98765432168', '1995-01-01'),
('Matriz', 1, '88866555576', '1995-01-01');
*/

INSERT INTO DEPARTAMENTO (Nome_departamento, Numero_departamento,
Cpf_gerente, Data_inicio_gerente)
VALUES
('Pequisa', 5, NULL, '1988-05-22'),
```

```
('Administração', 4, NULL, '1995-01-01'),
('Matriz', 1, NULL, '1995-01-01');

INSERT INTO FUNCIONARIO (Primeiro_Nome, Nome_meio, Ultimo_nome, Cpf,
Data_nascimento, Endereco, Sexo, Salario, Cpf_supervisor,
Numero_departamento)
VALUES
('João', 'B', 'Silva', '12345678966', '1965-01-09', 'Rua das Flores, 751. São Paulo,
SP', 'M', 30000, '33344555587', 5),
('Fernando', 'T', 'Wong', '33344555587', '1955-12-08', 'Rua da Lapa, 34, São
Paulo, SP', 'M', 40000, '88866555576', 5),
('Alice', 'J', 'Zelaya', '99988777767', '1968-01-19', 'Rua Souza Uma, 35, Curitiba,
PR', 'F', 25000, '98765432168', 4),
('Jennifer', 'S', 'Souza', '98765432168', '1941-06-20', 'Av. Arthur de Uma, 54, Santo
André, SP', 'F', 43000, '88866555576', 4),
('Ronaldo', 'K', 'Lima', '66688444476', '1962-09-15', 'Rua Rebouças, 65,
Piracicaba, SP', 'M', 38000, '33344555587', 5),
('Joice', 'A', 'Leite', '45345345376', '1972-07-31', 'Av. Lucas Obes, 74, São fóulo,
SP', 'F', 25000, '33344555587', 5),
('André', 'V', 'Pereira', '98798798733', '1969-03-29', 'Rua Timbira, 35, São Paulo,
SP', 'M', 25000, '98765432168', 4),
('Jorge', 'E', 'Brito', '88866555576', '1937-11-10', 'Rua do Horto, 35, São Paulo, SP',
'M', 55000, NULL, 1);

/*
<update statement> ::=
UPDATE <table name>
SET <set clause list>
[WHERE <search condition>]
*/

UPDATE departamento
SET Cpf_gerente = '33344555587'
WHERE Numero_departamento = 5;

UPDATE departamento
SET Cpf_gerente = '98765432168'
WHERE Numero_departamento = 4;
```

```
UPDATE departamento
SET Cpf_gerente = '88866555576'
WHERE Numero_departamento = 1;

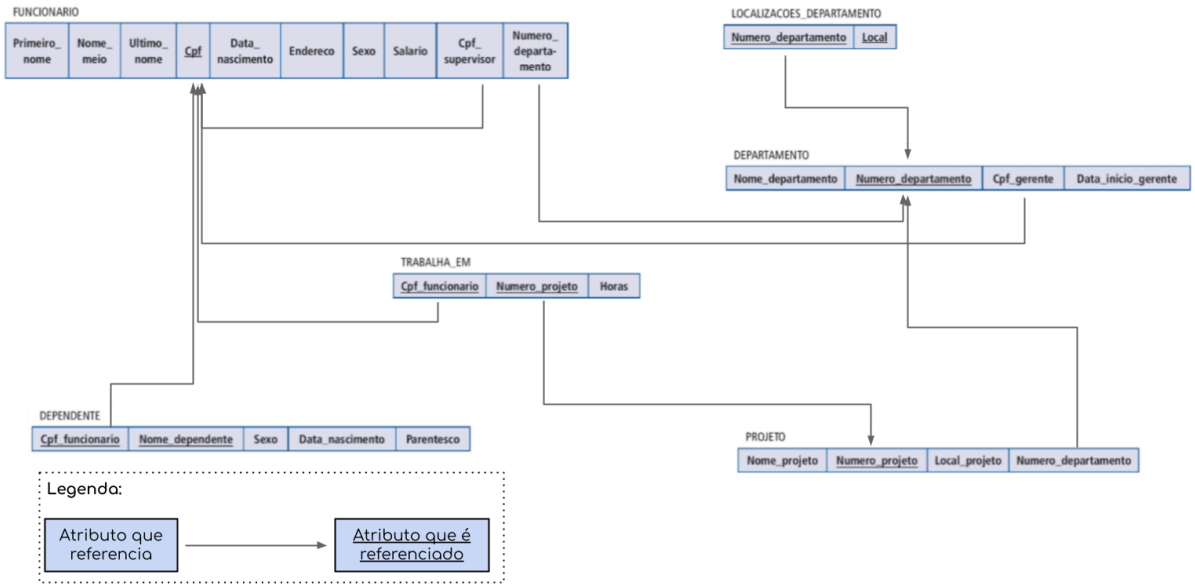
INSERT INTO LOCALIZACOES_DEPARTAMENTO (Numero_departamento, Local)
VALUES
(1, 'São Paulo'),
(4, 'Mauá'),
(5, 'Santo André'),
(5, 'Itu'),
(5, 'São Paulo');

INSERT INTO DEPENDENTE (Cpf_funcionario, Nome_dependente, Sexo,
Data_nascimento, Parentesco)
VALUES
('33344555587', 'Alicia', 'F', '1986-04-05', 'Filha'),
('33344555587', 'Tiago', 'M', '1983-10-25', 'Filho'),
('33344555587', 'Janaína', 'F', '1958-05-03', 'Esposa'),
('98765432168', 'António', 'M', '1942-02-28', 'Marido'),
('12345678966', 'Michael', 'M', '1988-01-04', 'Filho'),
('12345678966', 'Alícia', 'F', '1988-12-30', 'Filha'),
('12345678966', 'Elizabeth', 'F', '1967-05-05', 'Esposa');

INSERT INTO PROJETO (Nome_projeto, Numero_projeto, Local_projeto,
Numero_departamento)
VALUES
('ProdutoX', 1, 'Santo André', 5),
('ProdutoY', 2, 'Itu', 5),
('ProdutoZ', 3, 'São Paulo', 5),
('Informatização', 10, 'Mauá', 4),
('Reorganização', 20, 'São Paulo', 1),
('Novosbenefícios', 30, 'Mauá', 4);

INSERT INTO TRABALHA_EM (Cpf_funcionario, Numero_projeto, Horas)
VALUES
('12345678966', 1, 32.5),
('12345678966', 2, 7.5),
('66688444476', 3, 40.0),
```

('45345345376', 1, 20.0),
('45345345376', 2, 20.0),
('33344555587', 2, 10.0),
('33344555587', 3, 10.0),
('33344555587', 10, 10.0),
('33344555587', 20, 10.0),
('99988777767', 30, 30.0),
('99988777767', 10, 10.0),
('98798798733', 10, 35.0),
('98798798733', 30, 5.0),
('98765432168', 30, 20.0),
('98765432168', 20, 15.0),
('88866555576', 20, NULL);



3. Criando o modelo físico (SQL-DDL)

3.1. DDL: alternativa 1

00_sql_ddl_EMPRESA_Navathe_1.sql

discutir o motivo do erro (criar as constraints durante o ddl pode levar a uma dependencia cruzada).

1. Tabela `FUNCIONARIO`

```
CREATE TABLE FUNCIONARIO (
    Primeiro_Nome VARCHAR(50) NOT NULL,
    Nome_meio VARCHAR(50),
    Ultimo_nome VARCHAR(50) NOT NULL,
    Cpf CHAR(11) PRIMARY KEY,
    Data_nascimento DATE,
    Endereco VARCHAR(100),
    Sexo CHAR(1),
    Salario DECIMAL(10, 2),
    Cpf_supervisor CHAR(11),
    Numero_departamento INT,
    FOREIGN KEY (Cpf_supervisor) REFERENCES FUNCIONARIO(Cpf),
    FOREIGN KEY (Numero_departamento) REFERENCES DEPARTAMENTO
(Numero_departamento)
);
```

2. Tabela `DEPARTAMENTO`

```
CREATE TABLE DEPARTAMENTO (
    Nome_departamento VARCHAR(50) NOT NULL,
    Numero_departamento INT PRIMARY KEY,
```

```
    Cpf_gerente CHAR(11),
    Data_inicio_gerente DATE,
    FOREIGN KEY (Cpf_gerente) REFERENCES FUNCIONARIO(Cpf)
);
```

3. Tabela `LOCALIZACOES_DEPARTAMENTO`

```
CREATE TABLE LOCALIZACOES_DEPARTAMENTO (
    Numero_departamento INT,
    Local VARCHAR(50),
    FOREIGN KEY (Numero_departamento) REFERENCES DEPARTAMENTO
(Numero_departamento)
);
```

4. Tabela `DEPENDENTE`

```
CREATE TABLE DEPENDENTE (
    Cpf_funcionario CHAR(11),
    Nome_dependente VARCHAR(50),
    Sexo CHAR(1),
    Data_nascimento DATE,
    Parentesco VARCHAR(50),
    FOREIGN KEY (Cpf_funcionario) REFERENCES FUNCIONARIO(Cpf)
);
```

5. Tabela `PROJETO`

```
CREATE TABLE PROJETO (
    Nome_projeto VARCHAR(50),
    Numero_projeto INT PRIMARY KEY,
    Local_projeto VARCHAR(50),
```

```
    Numero_departamento INT,
    FOREIGN KEY (Numero_departamento) REFERENCES DEPARTAMENTO
(Numero_departamento)
);
```

6. Tabela `TRABALHA_EM`

```
CREATE TABLE TRABALHA_EM (
    Cpf_funcionario CHAR(11),
    Numero_projeto INT,
    Horas DECIMAL(4, 1),
    FOREIGN KEY (Cpf_funcionario) REFERENCES FUNCIONARIO(Cp
f),
    FOREIGN KEY (Numero_projeto) REFERENCES PROJETO(Numero_pr
ojeto),
    PRIMARY KEY (Cpf_funcionario, Numero_projeto)
);
```

3.2. DDL: alternativa 2 - modularizando as constraints

1. Criação das Tabelas Sem *Constraints*

```
-- Tabela FUNCIONARIO
CREATE TABLE FUNCIONARIO (
    Primeiro_Nome VARCHAR(50) NOT NULL,
    Nome_meio VARCHAR(50),
    Ultimo_nome VARCHAR(50) NOT NULL,
    Cpf CHAR(11),
    Data_nascimento DATE,
    Endereco VARCHAR(100),
    Sexo CHAR(1),
    Salario DECIMAL(10, 2),
    Cpf_supervisor CHAR(11),
```

```
    Numero_departamento INT
);

-- Tabela DEPARTAMENTO
CREATE TABLE DEPARTAMENTO (
    Nome_departamento VARCHAR(50) NOT NULL,
    Numero_departamento INT,
    Cpf_gerente CHAR(11),
    Data_inicio_gerente DATE
);

-- Tabela LOCALIZACOES_DEPARTAMENTO
CREATE TABLE LOCALIZACOES_DEPARTAMENTO (
    Numero_departamento INT,
    Local VARCHAR(50)
);

-- Tabela DEPENDENTE
CREATE TABLE DEPENDENTE (
    Cpf_funcionario CHAR(11),
    Nome_dependente VARCHAR(50),
    Sexo CHAR(1),
    Data_nascimento DATE,
    Parentesco VARCHAR(50)
);

-- Tabela PROJETO
CREATE TABLE PROJETO (
    Nome_projeto VARCHAR(50),
    Numero_projeto INT,
    Local_projeto VARCHAR(50),
    Numero_departamento INT
);

-- Tabela TRABALHA_EM
CREATE TABLE TRABALHA_EM (
```

```
    Cpf_funcionario CHAR(11),
    Numero_projeto INT,
    Horas DECIMAL(4, 1)
);
```

2. Adicionando *Constraints* Separadamente

01_sql_ddl_EMPRESA_Navathe_2.sql

```
-- Adicionando Constraints para FUNCIONARIO
ALTER TABLE FUNCIONARIO
ADD CONSTRAINT PK_Funcionario PRIMARY KEY (Cpf);

ALTER TABLE FUNCIONARIO
ADD CONSTRAINT FK_Funcionario_Supervisor FOREIGN KEY (Cpf_sup
ervisor)
REFERENCES FUNCIONARIO(Cpf);

-- Adicionando Constraints para DEPARTAMENTO
ALTER TABLE DEPARTAMENTO
ADD CONSTRAINT PK_Departamento PRIMARY KEY (Numero_departamen
to);

-- Para executar esta constraint, a primary key de "departame
nto" deve ter sido estabelecida
ALTER TABLE FUNCIONARIO
ADD CONSTRAINT FK_Funcionario_Departamento FOREIGN KEY (Numer
o_departamento)
REFERENCES DEPARTAMENTO(Numero_departamento);
```

```sql
ALTER TABLE DEPARTAMENTO
ADD CONSTRAINT FK_Departamento_Gerente FOREIGN KEY (Cpf_gerente)
REFERENCES FUNCIONARIO(Cpf);

-- Adicionando Constraints para LOCALIZACOES_DEPARTAMENTO
ALTER TABLE LOCALIZACOES_DEPARTAMENTO
ADD CONSTRAINT FK_Localizacoes_Departamento FOREIGN KEY (Numero_departamento)
REFERENCES DEPARTAMENTO(Numero_departamento);

-- Adicionando Constraints para DEPENDENTE
ALTER TABLE DEPENDENTE
ADD CONSTRAINT FK_Dependente_Funcionario FOREIGN KEY (Cpf_funcionario)
REFERENCES FUNCIONARIO(Cpf);

-- Adicionando Constraints para PROJETO
ALTER TABLE PROJETO
ADD CONSTRAINT PK_Projeto PRIMARY KEY (Numero_projeto);

ALTER TABLE PROJETO
ADD CONSTRAINT FK_Projeto_Departamento FOREIGN KEY (Numero_departamento)
REFERENCES DEPARTAMENTO(Numero_departamento);

-- Adicionando Constraints para TRABALHA_EM
ALTER TABLE TRABALHA_EM
ADD CONSTRAINT PK_Trabalha_Em PRIMARY KEY (Cpf_funcionario, Numero_projeto);

ALTER TABLE TRABALHA_EM
ADD CONSTRAINT FK_TrabalhaEm_Funcionario FOREIGN KEY (Cpf_funcionario)
REFERENCES FUNCIONARIO(Cpf);
```

```sql
ALTER TABLE TRABALHA_EM
ADD CONSTRAINT FK_TrabalhaEm_Projeto FOREIGN KEY (Numero_proj
eto)
REFERENCES PROJETO(Numero_projeto);
```

4. DML - Inserindo dados

02_sql_dml_EMPRESA_Navathe_3.sql

```sql
/*
Com erro
INSERT INTO DEPARTAMENTO (Nome_departamento, Numero_departame
nto, Cpf_gerente, Data_inicio_gerente)
VALUES
('Pequisa', 5, '33344555587', '1988-05-22'),
('Administração', 4, '98765432168', '1995-01-01'),
('Matriz', 1, '88866555576', '1995-01-01');
*/


INSERT INTO DEPARTAMENTO (Nome_departamento, Numero_departame
nto, Cpf_gerente, Data_inicio_gerente)
VALUES
('Pequisa', 5, NULL, '1988-05-22'),
('Administração', 4, NULL, '1995-01-01'),
('Matriz', 1, NULL, '1995-01-01');
```

```sql
INSERT INTO FUNCIONARIO (Primeiro_Nome, Nome_meio, Ultimo_nom
e, Cpf, Data_nascimento, Endereco, Sexo, Salario, Cpf_supervi
sor, Numero_departamento)
VALUES
('João', 'B', 'Silva', '12345678966', '1965-01-09', 'Rua das
Flores, 751. São Paulo, SP', 'M', 30000, '33344555587', 5),
('Fernando', 'T', 'Wong', '33344555587', '1955-12-08', 'Rua d
a Lapa, 34, São Paulo, SP', 'M', 40000, '88866555576', 5),
('Alice', 'J', 'Zelaya', '99988777767', '1968-01-19', 'Rua So
uza Uma, 35, Curitiba, PR', 'F', 25000, '98765432168', 4),
('Jennifer', 'S', 'Souza', '98765432168', '1941-06-20', 'Av.
Arthur de Uma, 54, Santo André, SP', 'F', 43000, '8886655557
6', 4),
('Ronaldo', 'K', 'Lima', '66688444476', '1962-09-15', 'Rua Re
bouças, 65, Piracicaba, SP', 'M', 38000, '33344555587', 5),
('Joice', 'A', 'Leite', '45345345376', '1972-07-31', 'Av. Luc
as Obes, 74, São fóulo, SP', 'F', 25000, '33344555587', 5),
('André', 'V', 'Pereira', '98798798733', '1969-03-29', 'Rua T
imbira, 35, São Paulo, SP', 'M', 25000, '98765432168', 4),
('Jorge', 'E', 'Brito', '88866555576', '1937-11-10', 'Rua do
Horto, 35, São Paulo, SP', 'M', 55000, NULL, 1);

/*
<update statement> ::=
  UPDATE <table name>
     SET <set clause list>
  [WHERE <search condition>]
*/

UPDATE departamento
SET Cpf_gerente = '33344555587'
WHERE Numero_departamento = 5;

UPDATE departamento
SET Cpf_gerente = '98765432168'
```

```sql
WHERE Numero_departamento = 4;

UPDATE departamento
SET Cpf_gerente = '88866555576'
WHERE Numero_departamento = 1;

INSERT INTO LOCALIZACOES_DEPARTAMENTO (Numero_departamento, L
ocal)
VALUES
(1, 'São Paulo'),
(4, 'Mauá'),
(5, 'Santo André'),
(5, 'Itu'),
(5, 'São Paulo');


INSERT INTO DEPENDENTE (Cpf_funcionario, Nome_dependente, Sex
o, Data_nascimento, Parentesco)
VALUES
('33344555587', 'Alicia', 'F', '1986-04-05', 'Filha'),
('33344555587', 'Tiago', 'M', '1983-10-25', 'Filho'),
('33344555587', 'Janaína', 'F', '1958-05-03', 'Esposa'),
('98765432168', 'António', 'M', '1942-02-28', 'Marido'),
('12345678966', 'Michael', 'M', '1988-01-04', 'Filho'),
('12345678966', 'Alícia', 'F', '1988-12-30', 'Filha'),
('12345678966', 'Elizabeth', 'F', '1967-05-05', 'Esposa');


INSERT INTO PROJETO (Nome_projeto, Numero_projeto, Local_proj
eto, Numero_departamento)
VALUES
('ProdutoX', 1, 'Santo André', 5),
('ProdutoY', 2, 'Itu', 5),
('ProdutoZ', 3, 'São Paulo', 5),
('Informatização', 10, 'Mauá', 4),
('Reorganização', 20, 'São Paulo', 1),
```

```
('Novosbenefícios', 30, 'Mauá', 4);


INSERT INTO TRABALHA_EM (Cpf_funcionario, Numero_projeto, Hor
as)
VALUES
('12345678966', 1, 32.5),
('12345678966', 2, 7.5),
('66688444476', 3, 40.0),
('45345345376', 1, 20.0),
('45345345376', 2, 20.0),
('33344555587', 2, 10.0),
('33344555587', 3, 10.0),
('33344555587', 10, 10.0),
('33344555587', 20, 10.0),
('99988777767', 30, 30.0),
('99988777767', 10, 10.0),
('98798798733', 10, 35.0),
('98798798733', 30, 5.0),
('98765432168', 30, 20.0),
('98765432168', 20, 15.0),
('88866555576', 20, NULL);
```

5. DML - primeiras consultas

Descrição	Consulta
Retorna todos os registros da tabela FUNCIONARIO.	SELECT * FROM FUNCIONARIO;
Retorna apenas os nomes e salários dos funcionários.	SELECT Primeiro_Nome, Ultimo_nome, Salario FROM FUNCIONARIO;
Seleciona todos os funcionários cujo salário é maior que 3000.	SELECT * FROM FUNCIONARIO WHERE Salario > 3000;

Conta o total de funcionários na tabela FUNCIONARIO.	SELECT COUNT(*) AS Total_Funcionarios FROM FUNCIONARIO;
Retorna todos os funcionários que pertencem ao departamento com o número 1.	SELECT * FROM FUNCIONARIO WHERE Numero_departamento = 1;
Retorna todos os dependentes do funcionário com o CPF especificado.	SELECT * FROM DEPENDENTE WHERE Cpf_funcionario = '12345678901';
Retorna o nome de cada departamento junto com o CPF do seu gerente.	SELECT Nome_departamento, Cpf_gerente FROM DEPARTAMENTO;
Calcula a média salarial de todos os funcionários.	SELECT AVG(Salario) AS Media_Salarial FROM FUNCIONARIO;
Lista os funcionários que têm dependentes.	SELECT DISTINCT F.Primeiro_Nome, F.Ultimo_nome FROM FUNCIONARIO F JOIN DEPENDENTE D ON F.Cpf = D.Cpf_funcionario;
Conta quantos dependentes cada funcionário tem.	SELECT Cpf_funcionario, COUNT(*) AS Total_Dependentes FROM DEPENDENTE GROUP BY Cpf_funcionario;
Retorna todos os funcionários, ordenados pelo salário em ordem decrescente.	SELECT * FROM FUNCIONARIO ORDER BY Salario DESC;
Mostra todos os projetos junto com o departamento ao qual pertencem.	SELECT P.Nome_projeto, D.Nome_departamento FROM PROJETO P JOIN DEPARTAMENTO D ON P.Numero_departamento = D.Numero_departamento;
Seleciona todos os funcionários cujo salário está acima da média salarial.	SELECT * FROM FUNCIONARIO WHERE Salario > (SELECT AVG(Salario) FROM FUNCIONARIO);
Agrupa e soma as horas trabalhadas em cada projeto.	SELECT Numero_projeto, SUM(Horas) AS Total_Horas FROM TRABALHA_EM GROUP BY Numero_projeto;
Lista todos os departamentos com suas respectivas localizações.	SELECT D.Nome_departamento, L.Local FROM DEPARTAMENTO D JOIN LOCALIZACOES_DEPARTAMENTO L ON D.Numero_departamento = L.Numero_departamento;
Mostra cada funcionário e o nome do seu supervisor.	SELECT F.Primeiro_Nome AS Funcionario, S.Primeiro_Nome AS Supervisor FROM

	FUNCIONARIO F LEFT JOIN FUNCIONARIO S ON F.Cpf_supervisor = S.Cpf;
Seleciona todos os departamentos que não têm gerente.	SELECT * FROM DEPARTAMENTO WHERE Cpf_gerente IS NULL;
Lista todos os funcionários e a quantidade de dependentes que cada um tem.	SELECT F.Primeiro_Nome, F.Ultimo_nome, COUNT(D.Cpf_funcionario) AS Total_Dependentes FROM FUNCIONARIO F LEFT JOIN DEPENDENTE D ON F.Cpf = D.Cpf_funcionario GROUP BY F.Cpf;
Retorna projetos onde o total de horas trabalhadas excede 50.	SELECT Numero_projeto, SUM(Horas) AS Total_Horas FROM TRABALHA_EM GROUP BY Numero_projeto HAVING SUM(Horas) > 50;
Retorna o salário máximo de funcionários em cada departamento.	SELECT Numero_departamento, MAX(Salario) AS Salario_Maximo FROM FUNCIONARIO GROUP BY Numero_departamento;
Lista funcionários cujo salário é igual ao salário do gerente do seu departamento.	SELECT F.Primeiro_Nome, F.Ultimo_nome FROM FUNCIONARIO F WHERE Salario = (SELECT Salario FROM FUNCIONARIO WHERE Cpf = (SELECT Cpf_gerente FROM DEPARTAMENTO WHERE Numero_departamento = F.Numero_departamento));
Retorna funcionários que têm dependentes com parentesco 'filho'.	SELECT F.Primeiro_Nome, F.Ultimo_nome FROM FUNCIONARIO F WHERE F.Cpf IN (SELECT D.Cpf_funcionario FROM DEPENDENTE D WHERE D.Parentesco = 'filho');
Lista departamentos que têm pelo menos um funcionário associado.	SELECT D.Nome_departamento FROM DEPARTAMENTO D WHERE D.Numero_departamento IN (SELECT DISTINCT Numero_departamento FROM FUNCIONARIO);
Retorna projetos que têm menos de 10 horas trabalhadas.	SELECT Numero_projeto FROM PROJETO WHERE Numero_projeto NOT IN (SELECT Numero_projeto FROM TRABALHA_EM GROUP BY Numero_projeto HAVING SUM(Horas) >= 10);

* Lista funcionários que estão envolvidos em mais de um projeto.	SELECT Cpf_funcionario, COUNT(Numero_projeto) AS Projetos_Ativos FROM TRABALHA_EM GROUP BY Cpf_funcionario HAVING COUNT(Numero_projeto) > 1;
Retorna todos os funcionários com seus salários e um ranking baseado nos salários.	SELECT Primeiro_Nome, Ultimo_nome, Salario, RANK() OVER (ORDER BY Salario DESC) AS Salario_Rank FROM FUNCIONARIO;
Mostra a soma acumulada de horas trabalhadas por cada funcionário em projetos.	SELECT Cpf_funcionario, Numero_projeto, Horas, SUM(Horas) OVER (PARTITION BY Cpf_funcionario ORDER BY Numero_projeto) AS Soma_Acumulada FROM TRABALHA_EM;
Calcula a média de horas trabalhadas por projeto.	SELECT Numero_projeto, AVG(Horas) AS Media_Horas FROM TRABALHA_EM GROUP BY Numero_projeto;
Lista funcionários que ganham acima da média salarial, juntamente com seu ranking.	SELECT Primeiro_Nome, Ultimo_nome, Salario, RANK() OVER (ORDER BY Salario DESC) AS Salario_Rank FROM FUNCIONARIO WHERE Salario > (SELECT AVG(Salario) FROM FUNCIONARIO);
* Retorna o total de salários pagos em cada departamento.	SELECT D.Nome_departamento, SUM(F.Salario) AS Total_Salarios FROM DEPARTAMENTO D JOIN FUNCIONARIO F ON D.Numero_departamento = F.Numero_departamento GROUP BY D.Nome_departamento;
listar funcionários que trabalharam mais de 20 horas em projetos	SELECT f.primeiro_nome AS nome_funcionario, f.ultimo_nome AS sobrenome_funcionario, d.nome_departamento, p.nome_projeto, te.horas AS horas_trabalhadas FROM funcionario f INNER JOIN trabalha_em te ON f.cpf = te.cpf_funcionario

	INNER JOIN projeto p ON te.numero_projeto = p.numero_projeto INNER JOIN departamento d ON f.numero_departamento = d.numero_departamento WHERE te.horas > 20 ORDER BY horas_trabalhadas DESC;
	SELECT f.primeiro_nome AS nome_funcionario, f.ultimo_nome AS sobrenome_funcionario, (SELECT d.nome_departamento FROM departamento d WHERE d.numero_departamento = f.numero_departamento) AS nome_departamento, (SELECT p.nome_projeto FROM projeto p WHERE p.numero_projeto = te.numero_projeto) AS nome_projeto, te.horas AS horas_trabalhadas FROM funcionario f INNER JOIN trabalha_em te ON f.cpf = te.cpf_funcionario WHERE te.horas > 20 ORDER BY horas_trabalhadas DESC;

dump16-10-25.sql

s.-+

6. ‘Views’ e ‘Materialized Views’

Views

Ver mais em: https://www.postgresql.org/docs/17/sql-createview.html

> 💡 VIEWS definem a visão de uma consulta e não são fisicamente armazenadas (materializadas). A consulta é executada todas as vezes em que for chamada.

```
CREATE [ OR REPLACE ] [ TEMP | TEMPORARY ] [ RECURSIVE ] VIEW
name [ ( column_name [, ...] ) ]
    [ WITH ( view_option_name [= view_option_value] [, ... ]
) ]
    AS query
    [ WITH [ CASCADED | LOCAL ] CHECK OPTION ]
```

https://www.postgresql.org/docs/17/sql-createview.html

Considere as seguintes consultas:

```
--  Q1: Retorna todos os funcionários que pertencem ao depart
amento com o número 1.
SELECT * FROM FUNCIONARIO WHERE Numero_departamento = 1;

-- Q2: Retorna o nome de cada departamento junto com o CPF do
seu gerente.
SELECT Nome_departamento, Cpf_gerente FROM DEPARTAMENTO;
```

```sql
-- Q3: Lista todos os funcionários e a quantidade de dependen
tes que cada um tem.
SELECT F.Primeiro_Nome,
       F.Ultimo_nome,
       COUNT(D.Cpf_funcionario) AS Total_Dependentes
    FROM FUNCIONARIO F LEFT JOIN DEPENDENTE D
        ON F.Cpf = D.Cpf_funcionario GROUP BY F.Cpf;
```

CREATE VIEW

```sql
-- VIEW Q1
CREATE VIEW vw_func_dpto AS SELECT * FROM FUNCIONARIO WHERE N
umero_departamento = 1;
SELECT * FROM vw_func_dpto;
```

Caso haja a tentativa de criar uma view de mesmo nome, o banco de dados retornará o seguinte erro: `ERROR:  relation "vw_func_dpto" already exists`

CREATE OR REPLACE VIEW

```sql
-- VIEW Q2
CREATE OR REPLACE VIEW vw_dpto_gerente AS SELECT Nome_departa
mento, Cpf_gerente FROM DEPARTAMENTO;
SELECT * FROM vw_dpto_gerente;
```

PARAMETER 'TEMPORARY' or 'TEMP'

```sql
-- VIEW Q3
CREATE TEMPORARY VIEW vw_tmp_func_dep AS SELECT

F.Primeiro_Nome,

F.Ultimo_nome,

COUNT(D.Cpf_funcionario) AS Total_Dependentes
                                                    FROM
FUNCIONARIO F LEFT JOIN DEPENDENTE D
                                                    ON F.
Cpf = D.Cpf_funcionario GROUP BY F.Cpf;

SELECT * FROM vw_tmp_func_dep;
```

A VIEW `vw_tmp_func_dep` será apagada quando a conexão com o banco for cancelada ou fechada.

PARAMETER 'WITH' [CASCADED | LOCAL] CHECK OPTION

LOCAL vai permitir alterações na VIEW `vw_dpto_gerente` mesmo quando as regras de busca de sua criação não forem respeitadas.

Ver mais em: https://www.ibm.com/docs/en/i/7.3?topic=view-local-check-option

```sql
-- VIEW Q2.3
CREATE OR REPLACE VIEW vw_dpto_gerente AS
    SELECT Nome_departamento, Cpf_gerente FROM DEPARTAMENTO
    WITH LOCAL CHECK OPTION;

SELECT * FROM vw_dpto_gerente;
```

CASCADE só vai permitir as alterações na VIEW `vw_dpto_gerente` que correspondem às regras de sua criação (em cascata), considerando suas dependencias.

Ver mais em: https://www.ibm.com/docs/en/i/7.3?topic=view-cascaded-check-option

```
-- VIEW Q2.3
CREATE OR REPLACE VIEW vw_dpto_gerente AS
    SELECT Nome_departamento, Cpf_gerente FROM DEPARTAMENTO
    WITH CASCADED CHECK OPTION;

SELECT * FROM vw_dpto_gerente;
```

Materialized Views

Ver mais em: https://www.postgresql.org/docs/current/sql-creatematerializedview.html



Uma visão materializada é similar a uma tabela. Portanto os comandos 'CREATE MATERIALIZED VIEW' é equivalente ao 'CREATE TABLE AS'. A primeira opção, contudo, armazena o comando usado para a sua inicialização para permitir sua atualização sob demanda. Não existe MATERIALIZED VIEW temporária.

```
CREATE MATERIALIZED VIEW [ IF NOT EXISTS ] table_name
    [ (column_name [, ...] ) ]
    [ USING method ]
    [ WITH ( storage_parameter [= value] [, ... ] ) ]
    [ TABLESPACE tablespace_name ]
```

```
    AS query
    [ WITH [ NO ] DATA ]
```

CREATE MATERIALIZED VIEW IF NOT EXISTS

```
-- Q3.3.MV
CREATE MATERIALIZED VIEW IF NOT EXISTS mvw_func_dep
AS SELECT F.Primeiro_Nome,
       F.Ultimo_nome,
       COUNT(D.Cpf_funcionario) AS Total_Dependentes
    FROM FUNCIONARIO F LEFT JOIN DEPENDENTE D
        ON F.Cpf = D.Cpf_funcionario GROUP BY F.Cpf;
```

REFRESH MATERIALIZED VIEW

Ver mais em: https://www.postgresql.org/docs/current/sql-refreshmaterializedview.html



O comando 'REFRESH MATERIALIZED VIEW' substitui completamente uma MATERIALIZED VIEW estabelecendo (ou não) o estado de escaneável através da flag [WITH [NO] DATA]. Caso o 'WITH NO DATA' tenha sido usado, a tabela será limpa, o armazenamento liberado, e este estado só mudará com um novo refresh sem esta clausula.

```
REFRESH MATERIALIZED VIEW [ CONCURRENTLY ] name
    [ WITH [ NO ] DATA ]
```

Exemplo:

```
REFRESH MATERIALIZED VIEW mvw_func_dep;
```

7. Stored procedures

CREATE PROCEDURE

Ver mais: https://www.postgresql.org/docs/current/sql-createprocedure.html

💡 Com o CREATE OR REPLACE PROCEDURE é possível construir um conjunto de passos atomicos para cumprir com uma tarefa.

```
CREATE [ OR REPLACE ] PROCEDURE
    name ( [ [ argmode ] [ argname ] argtype [ { DEFAULT | =
} default_expr ] [, ...] ] )
  { LANGUAGE lang_name
    | TRANSFORM { FOR TYPE type_name } [, ... ]
    | [ EXTERNAL ] SECURITY INVOKER | [ EXTERNAL ] SECURITY D
EFINER
    | SET configuration_parameter { TO value | = value | FROM
CURRENT }
    | AS 'definition'
    | AS 'obj_file', 'link_symbol'
    | sql_body
  } ...
```

Exemplo

```
CREATE OR REPLACE PROCEDURE inserir_dptp_loc(ndpt integer,

nmdpt varchar,

cpf_ger char,

dt_ini_ger date,

locdpt varchar)
LANGUAGE SQL
BEGIN ATOMIC
  INSERT INTO DEPARTAMENTO VALUES (nmdpt, ndpt, cpf_ger, dt_i
ni_ger);
  INSERT INTO LOCALIZACOES_DEPARTAMENTO VALUES (ndpt, locdp
t);
END;
```

CALL

Ver mais: https://www.postgresql.org/docs/current/sql-call.html

```
CALL inserir_dptp_loc('7',
                                        'Depto Teste',
                                        '88866555576',
                                        '1987-05-06',
                                        'Na salinha anexa
ao almoxarifado');
```

```
SELECT * FROM DEPARTAMENTO;
SELECT * FROM LOCALIZACOES_DEPARTAMENTO;
```

Transações

https://www.postgresql.org/docs/current/tutorial-transactions.html

Demonstrando a exclusão mútua

Em suas seções PSQL, execute os seguintes comandos:

Seção 1:

```
BEGIN;
UPDATE departamento SET Cpf_gerente = '33344555587' WHERE Num
ero_departamento = 5;
```

Seção 2 (sem commit na seção 1):

```
BEGIN;
UPDATE departamento SET Cpf_gerente = '99988777767' WHERE Num
ero_departamento = 5;
```

Definindo o escalonamento:

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

```
UPDATE departamento SET Cpf_gerente = '33344555587' WHERE Num
ero_departamento = 5;
```

Pontos de salvamento

```
BEGIN;
SAVEPOINT antes_da_transacao;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
UPDATE departamento SET Cpf_gerente = '33344555587' WHERE Num
ero_departamento = 5;
ROLLBACK TO antes_da_transacao;
COMMIT;
```